Examen — Codage et traitement du signal

Clones fonctionnels des utilitaires de compression bzip2, lzma et 7-zip

Étudiant : **RALISON Andriniaina Stephano Michel**

Date limite de remise : samedi 24 octobre 2026, 23h59

Langage : Python 3 (bibliothèque standard uniquement ; les algorithmes de compression sont réimplémentés, `zlib.crc32` sert seulement de somme de contrôle).

Dépôt GitHub : https://github.com/stephanoralison8/Clone-compress

1. Principe des trois outils

bzip2-clone — mêmes étapes que bzip2 : pré-codage RLE, transformée de Burrows-Wheeler (qui regroupe les octets semblables), Move-To-Front, codage des plages de zéros, puis codage de Huffman (codes courts pour les octets fréquents), par blocs.

lzma-clone — compression par dictionnaire LZ77 (une répétition est remplacée par un renvoi « même chose que N octets plus tôt, sur L octets ») suivie d'un codeur par intervalles binaire adaptatif, comme LZMA.

7z-clone — archiveur : plusieurs fichiers sont regroupés en un flux unique compressé avec le moteur lzma-clone ; l'entête (noms, tailles, dates, CRC32) est stocké avec l'archive. Commandes : `a` (créer), `l` (lister), `t` (tester), `x` (extraire).

Les formats produits (.bzc, .lzc, .7zc) sont propres au projet ; l'absence de perte est vérifiée par CRC32 et par comparaison SHA-256.

2. Résultats sur quatre types de fichiers

Type	Fichier	Original	Outil	Compressé	% de l'original	Gain
Texte	texte.txt	307,68 Ko	bzip2-clone	49,72 Ko	16,16 %	83,84 %
Audio	audio.wav	258,44 Ko	lzma-clone	239,82 Ko	92,79 %	7,21 %
Image	image.bmp	225,05 Ko	lzma-clone	101,43 Ko	45,07 %	54,93 %
Vidéo	video.y4m	562,66 Ko	bzip2-clone	4,53 Ko	0,81 %	99,19 %

Archive 7z-clone (les 4 fichiers ensemble) : 1,32 Mo vers 419,73 Ko, soit 31,00 % de l'original.

Analyse. Les gains dépendent de la redondance des données. La vidéo brute (images presque identiques d'une image à l'autre) et le texte (mots répétés) se compressent très bien. L'image (aplats et dégradés) se compresse moyennement. L'audio contient du bruit de fond, presque incompressible sans perte : le gain est faible. Un fichier déjà compressé (MP3, JPEG, MP4) ne gagnerait presque rien.

3. Captures d'écran

3.1 Compression des quatre types de fichiers

```
PS C:\L2IDEV\Codage_traitement_signal\clone_compression> py clonecompress.py bzip2 -k -f -v echantillons/texte.txt 2>$null
>> py clonecompress.py lzma -k -f -v echantillons/audio.wav 2>$null
>> py clonecompress.py lzma -k -f -v echantillons/image.bmp 2>$null
>> py clonecompress.py bzip2 -k -f -v echantillons/video.y4m 2>$null
echantillons/texte.txt -> echantillons/texte.txt.bzc : 307.68 Ko -> 49.72 Ko (16.16 % de l'original, gain 83.84 %) en 4.23 s
echantillons/audio.wav -> echantillons/audio.wav.lzc : 258.44 Ko -> 239.82 Ko (92.79 % de l'original, gain 7.21 %) en 7.62 s
echantillons/image.bmp -> echantillons/image.bmp.lzc : 225.05 Ko -> 101.43 Ko (45.07 % de l'original, gain 54.93 %) en 4.28 s
echantillons/video.y4m -> echantillons/video.y4m.bzc : 562.66 Ko -> 4.53 Ko (0.81 % de l'original, gain 99.19 %) en 4.39 s
PS C:\L2IDEV\Codage_traitement_signal\clone_compression> _
```

Capture 1 — Compression du texte (bzip2-clone), de l'audio et de l'image (lzma-clone) et de la vidéo (bzip2-clone), avec la taille avant/après et le gain obtenu.

3.2 Archive 7z-clone : création, liste et test

```
PS C:\L2IDEV\Codage_traitement_signal\clone_compression> py clonecompress.py 7z a demo.7zc echantillons/texte.txt echantillons/audio.w
av echantillons/image.bmp echantillons/video.y4m
>> py clonecompress.py 7z l demo.7zc
>> py clonecompress.py 7z t demo.7zc
Compression  98 %Archive demo.7zc créée : 4 fichier(s), 1.32 Mo -> 419.73 Ko (31.00 % de l'original)
Durée : 16.87 s
Date                 Taille     CRC32  Nom
-------------------------------------------------------------
2026-10-22 22:29        315062  b1a22e74  texte.txt
2026-10-22 22:29        264644  b35ee8a1  audio.wav
2026-10-22 22:29        230454  8af887f2  image.bmp
2026-10-22 22:29        576163  dbeddbbb  video.y4m
-------------------------------------------------------------
                       1386323  4 entrée(s)
Test OK   texte.txt
Test OK   audio.wav
Test OK   image.bmp
Test OK   video.y4m
Aucune erreur.
```

Capture 2 — Création de l'archive demo.7zc contenant les 4 fichiers (1,32 Mo vers 419,73 Ko), liste de son contenu (nom, taille, CRC32) puis test d'intégrité : « Aucune erreur ».

3.3 Vérification : aucune perte à la décompression

```
PS C:\L2IDEV\Codage_traitement_signal\clone_compression> py clonecompress.py lzma -d -k -f -v -o image_restauree.bmp echantillons/imag
e.bmp.lzc
>> Get-FileHash echantillons/image.bmp, image_restauree.bmp
>> (Get-FileHash echantillons/image.bmp).Hash -eq (Get-FileHash image_restauree.bmp).Hash
echantillons/image.bmp.lzc -> image_restauree.bmp : 101.43 Ko -> 225.05 Ko en 2.78 s

Algorithm       Hash                                                                   Path
---------       ----                                                                   ----
SHA256          491449995284B5EF7E16777117ED8144244E8E4923793B99CF4A3D2C24851C5A       C:\L2IDEV\Codage_traitement_signal\clone_co...
SHA256          491449995284B5EF7E16777117ED8144244E8E4923793B99CF4A3D2C24851C5A       C:\L2IDEV\Codage_traitement_signal\clone_co...
True
```

Capture 3 — Décompression de image.bmp.lzc puis comparaison SHA-256 avec l'original : les deux empreintes sont identiques et la comparaison renvoie « True ». La compression est donc sans perte.

4. Conclusion

Les trois clones compressent et décompressent sans perte les quatre types de fichiers testés. Les résultats sont conformes à la théorie de l'information : plus les données sont redondantes, plus le gain est grand. Les implémentations en Python sont plus lentes et un peu moins efficaces que les outils officiels (écrits en C), mais reprennent les mêmes principes.